

Annotated versus unannotated Russell universes for type theory with explicit universe polymorphism (BCDE)

Formalisation notes (`~/DOMAIN/BCDE4`)

28 September 2026

This note is the analogue, for the system of `bcde.pdf` (internal universe levels, level products $[\alpha]A$, constraint products $[\psi]A$, cumulative universes U_l à la Russell, App. B, and Gonthier’s collapse, App. C), of the notes `ERT/sec43.tex` on §4.3 of Sterbac’s paper. It records the annotated Russell system T_R , the unannotated system T_P (the paper’s own presentation), and the proofs of the analogues of Sterbac’s Theorem 4.15 (stripping), Lemma 4.18 (redundancy of annotations) and Theorem 4.19 (section of `strip`). Everything is in Agda with `--safe --without-K --exact-split`: no postulates, no termination pragmas. `BCDE4/All.agda` checks from scratch in about 35 s; no module takes more than 3.1 s.

Lemma 4.18 is proved *without normalisation*, by the η -coercion and join argument of `ERT/sec43.tex` §5, extended to level and constraint products. Its non-syntactic inputs are all proved from the finite-element domain model:

- Π -, $[\alpha]$ - and U -injectivity (adequacy);
- the no-confusion of U , Π and $[\alpha]$ (soundness);
- a guard-aware no-confusion for $[\psi]A$ (soundness).

One rule had to be added to the paper’s system: `guard  $\eta$` (§5).

As in the ERT notes, premises in `grey` are presuppositions: admissible, not in the paper, present only to keep the metatheory, the adequacy proof and the section of `strip` structurally recursive. Rules in `blue` go beyond `bcde.pdf`.

1 Syntax

Levels $l ::= \alpha_i \mid l \vee m \mid l^+$ (de Bruijn level variables), constraints $\psi ::= l = m$. A context carries a finite level theory at its bottom; Γ, ψ adds a constraint, Γ, α a fresh level variable (shifting the others). $l \leq m$ means $l \vee m = m$ and $l < m$ means $l^+ \leq m$. Γ has a loop if $l < l$ is valid in Γ for some l . Loops are decidable (Bezém–Coquand, `LC/Loop.decLoop`).

$$\begin{aligned} T_R : \quad A, a, \dots & ::= v_i \mid U_l \mid \Pi(A, B) \mid \lambda(A, B, b) \mid \text{app}(A, B, c, a) \mid [\psi]A \mid \langle \psi \rangle_A t \mid \emptyset \mid [\alpha]A \mid \langle \alpha \rangle_A u \mid t_A l \\ T_P : \quad A, a, \dots & ::= v_i \mid U_l \mid \Pi A B \mid \lambda A b \mid c a \mid [\psi]A \mid \langle \psi \rangle t \mid \emptyset \mid [\alpha]A \mid \langle \alpha \rangle u \mid t l \end{aligned}$$

In T_R the binders of level and constraint products are annotated by their type (`bcde.pdf`, Remark 2):

- $\langle \psi \rangle_A t : [\psi]A$;
- $\langle \alpha \rangle_A u : [\alpha]A$;
- $t_A l : A[l/\alpha]$ for $t : [\alpha]A$.

Without these annotations the Russell–Tarski uniqueness lemma is false: $\langle \psi \rangle(\uparrow^{n_0} v)$ and $\langle \psi \rangle(\uparrow^{n_1} v)$ erase alike but have types $[\psi]U_{n_0} \neq [\psi]U_{n_1}$. T_P is the core syntax of the domain model (`Model/Core.agda`), and

$$\text{strip}(\lambda(A, B, b)) = \lambda \text{strip} A \text{strip} b, \quad \text{strip}(\text{app}(A, B, c, a)) = \text{strip} c \text{strip} a,$$

$$\text{strip}(\langle \psi \rangle_A t) = \langle \psi \rangle \text{strip} t, \quad \text{strip}(\langle \alpha \rangle_A u) = \langle \alpha \rangle \text{strip} u, \quad \text{strip}(t_A l) = \text{strip} t l,$$

and homomorphically elsewhere. `strip` commutes with renaming, term substitution, level substitution and context lookup.

2 The system T_R (RussellTyping.agda)

Judgements $\vdash \Gamma$, $\Gamma \vdash A$ type, $\Gamma \vdash a : A$, $\Gamma \vdash A = B$, $\Gamma \vdash a = b : A$. The Π fragment (λ , **app**, their congruences, β , η) is as in `ERT/sec43.tex`. The universes follow App. B with cumulativity:

$$\frac{\vdash \Gamma \quad l < m}{\Gamma \vdash U_l : U_m} \quad \frac{\Gamma \vdash A : U_l \quad l \leq m}{\Gamma \vdash A : U_m} \quad \frac{\Gamma \vdash A : U_l \quad \Gamma.A \vdash B : U_l}{\Gamma \vdash \Pi(A, B) : U_l} \quad \frac{\vdash \Gamma}{\Gamma \vdash \emptyset : U_l}$$

$$\frac{\vdash \Gamma \quad l = l' \quad l < m}{\Gamma \vdash U_l = U_{l'} : U_m}$$

Types form a separate judgement. The products $[\alpha]A$ and $[\psi]A$ are *large*: they are types but live in no universe.

$$\frac{\Gamma \vdash A : U_l}{\Gamma \vdash A \text{ type}} \quad \frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type}}{\Gamma \vdash \Pi(A, B) \text{ type}} \quad \frac{\vdash \Gamma \quad \Gamma, \alpha \vdash A \text{ type}}{\Gamma \vdash [\alpha]A \text{ type}} \quad \frac{\vdash \Gamma \quad \Gamma, \psi \vdash A \text{ type}}{\Gamma \vdash [\psi]A \text{ type}}$$

Level products (Fig. 12).

$$\frac{\vdash \Gamma \quad \Gamma, \alpha \vdash A \text{ type} \quad \Gamma, \alpha \vdash u : A}{\Gamma \vdash \langle \alpha \rangle_A u : [\alpha]A} \quad \frac{\Gamma, \alpha \vdash A \text{ type} \quad \Gamma \vdash t : [\alpha]A}{\Gamma \vdash t_A l : A[l/\alpha]}$$

$$\frac{\Gamma, \alpha \vdash A \text{ type} \quad \Gamma, \alpha \vdash u : A}{\Gamma \vdash (\langle \alpha \rangle_A u)_A l = u[l/\alpha] : A[l/\alpha]} (\beta) \quad \frac{\Gamma, \alpha \vdash A \text{ type} \quad \Gamma \vdash t : [\alpha]A}{\Gamma \vdash t = \langle \alpha \rangle_A (t^\uparrow_{A^\uparrow} \alpha) : [\alpha]A} (\eta)$$

$$\frac{\Gamma, \alpha \vdash A \text{ type} \quad \Gamma \vdash t : [\alpha]A \quad l = l' \quad \Gamma \vdash A[l/\alpha] = A[l'/\alpha]}{\Gamma \vdash t_A l = t_A l' : A[l/\alpha]}$$

with congruences for $\langle \alpha \rangle$ (body and annotation) and for level application (function and annotation). Here $t^\uparrow$ shifts the level variables and $A^\uparrow$ does so under the binder.

Constraint products (Fig. 13).

$$\frac{\vdash \Gamma \quad \Gamma, \psi \vdash A \text{ type} \quad \Gamma, \psi \vdash t : A}{\Gamma \vdash \langle \psi \rangle_A t : [\psi]A} \quad \frac{\psi \text{ valid} \quad \Gamma \vdash A \text{ type}}{\Gamma \vdash [\psi]A = A} \quad \frac{\psi \text{ valid} \quad \Gamma \vdash A \text{ type} \quad \Gamma \vdash t : A}{\Gamma \vdash \langle \psi \rangle_A t = t : A}$$

$$\frac{\psi \Leftrightarrow \psi' \text{ valid} \quad \Gamma, \psi \vdash A \text{ type}}{\Gamma \vdash [\psi]A = [\psi']A} \quad \frac{\psi \Leftrightarrow \psi' \text{ valid} \quad \Gamma, \psi \vdash t : A}{\Gamma \vdash \langle \psi \rangle_A t = \langle \psi' \rangle_A t : [\psi]A}$$

$$\frac{\vdash \Gamma \quad \Gamma, \psi \vdash A \text{ type} \quad \Gamma \vdash t : [\psi]A \quad \Gamma, \psi \vdash t : A}{\Gamma \vdash t = \langle \psi \rangle_A t : [\psi]A} (\eta)$$

with congruences for $[\psi]$ and $\langle \psi \rangle$ (body and annotation). There is no eliminator for $[\psi]A$. A term of type $[\psi]A$ is used at type A only where ψ is valid, through $[\psi]A = A$.

Collapse (Fig. 14). If Γ has a loop: $\Gamma \vdash A = \emptyset$ for every type A , $\Gamma \vdash \emptyset : A$, and $\Gamma \vdash t = \emptyset : A$ for every $t : A$.

3 The system T_P (PTyping.agda)

The rules of T_P are those of T_R with the annotations removed, on T_P syntax: for instance $\Gamma \vdash \langle \psi \rangle t : [\psi]A$, $\Gamma \vdash t l : A[l/\alpha]$ for $t : [\alpha]A$, $(\langle \alpha \rangle u) l = u[l/\alpha]$, $t = \langle \alpha \rangle (t^\uparrow \alpha)$ and $t = \langle \psi \rangle t : [\psi]A$. In T_R the annotation congruences of **app**, $\langle \psi \rangle$, $\langle \alpha \rangle$ and level application have two sides with the same stripping, so they have no counterpart in T_P . The congruence for λ 's annotation keeps its domain part. Up to the presupposition premises and guard η , T_P is the Russell system of `bcde.pdf` (App. B, App. C).

4 Differences with the paper

1. **Annotations** of $\langle \psi \rangle$, $\langle \alpha \rangle$ and level application in T_R (Remark 2), besides those of λ and **app**.
2. **Presupposition premises** (grey). They are admissible: see the `mk-*` lemmas of `RussellMeta` and `RussellLeq`.

3. **Guard η** , in T_R , in T_T and in T_P . It is not in Fig. 13, and §5 explains why it is needed.
4. **Variables** are typed by lookup, and the congruences are split into one rule per argument, as in `ERT/sec43.tex`.

5 Guards: injectivity, no-confusion, and η

The naive statements are false. Guard β ($[\psi]A = A$ when ψ is valid) breaks both statements that one would transcribe from Π :

- $[\psi]A = [\psi']A'$ does *not* imply $\psi \Leftrightarrow \psi'$. With ψ valid (e.g. $\alpha = \alpha$), $[\psi][\psi']B = [\psi']B$ for every ψ' .
- $[\psi]A$ is not distinct from $\mathbf{U}$, Π , $[\alpha]$: $[\psi]\mathbf{U}_l = \mathbf{U}_l$ when ψ is valid.

What holds (`GrdNoConf.agda`).

Lemma 1. 1. $\Gamma \vdash [\psi]A = [\psi']A'$ implies $\Gamma, \psi, \psi' \vdash A = A'$. Hence $\Gamma \vdash [\psi]A = [\psi]A'$ implies $\Gamma, \psi \vdash A = A'$ (*grd-inj*, *grd-inj1*).

2. Let Γ be loop-free and let X be a universe, a Π or a $[\alpha]$. Then $\Gamma \vdash X = [\psi]A$ implies that ψ is valid in Γ (*grd-valid-U*, *grd-valid-Pi*, *grd-valid-LPi*).

3. In a loop-free context, universes, Π 's and $[\alpha]$'s are pairwise not convertible (*noconf-**).

Proof. (1) is purely syntactic. Weaken the conversion to Γ, ψ, ψ' , where both guards are valid, and apply guard β on both sides.

(2) and (3) come from soundness of the model at the bottom environment of Γ , whose ambient level theory is that of Γ . There $[\psi]A$ denotes $\{\perp\}$ when ψ is not valid. Meanwhile $\mathbf{U}_l$, Π and $[\alpha]$ contain the non- $\perp$ elements $\mathbf{U}$ -code, $\Pi(\perp, \emptyset)$ and $[\alpha]\emptyset$ respectively, and these three are pairwise distinct. $\square$

The consumers of Lemma 1 never compare a guard with a guard. They compare a guard with $\mathbf{U}$, Π or $[\alpha]$, where (2) forces the guard to be valid, so that it can be dropped.

Why η is needed. Under a guard, cumulativity is not subsumption: $\langle \psi \rangle_{\mathbf{U}_k} t : [\psi]\mathbf{U}_k$ is not of type $[\psi]\mathbf{U}_m$ for $k < m$. So the coercions of §8 must go through guards:

$$\text{coe}_{[\psi]A \leq [\psi]L}(t) = \langle \psi \rangle_L \text{coe}_{A \leq L}(t) \quad (t \text{ has type } A \text{ in } \Gamma, \psi \text{ by guard } \beta).$$

The diagonal lemma then has to go from $\Gamma, \psi \vdash u_0 = u_1 : A$ to $\Gamma \vdash u_0 = u_1 : [\psi]A$, and u_0, u_1 need not be $\langle \psi \rangle$ -abstractions: they can be β -redexes or neutral terms. Without guard η this step is not derivable in general. For example, take variables $x, y : [\psi]A$ with Γ, ψ loop-free. Then $\Gamma, \psi \vdash x = y$ holds by collapse, but no rule gives $\Gamma \vdash x = y : [\psi]A$ (this was checked by hand, not formalised).

Guard η is harmless.

- **Sound:** a term of type $[\psi]A$ denotes $\perp$ wherever ψ fails. Model case: `Model/RussellSound.InvConv-GLamEta`; adequacy case: `Adeq/Guard.AdqE1-GLam-eta`.
- **Cheap:** the metatheory, erasure and Russell–Tarski (Theorem 4.13) only needed mechanical new cases.
- **Natural:** $[\psi]A$ is a product over the proposition ψ , and Π and $[\alpha]$ already have η . Moreover, when Γ, ψ loops, η makes $[\psi]A$ a definitional unit type: $x = \langle \psi \rangle x = \langle \psi \rangle y = y$. This is the “unit type” solution discussed in App. C, obtained here without extra syntax.

6 What is proved from the model

Theorem 2 (*Adeq/Driver, Main*). *The finite-element model with levelled universe codes (level codes by the Bezem–Coquand decision procedure) is adequate for T_R . Hence, in loop-free contexts:*

- Π -injectivity: $\Pi(C_0, B_0) = \Pi(C_1, B_1) \Rightarrow C_0 = C_1$ and $B_0 = B_1$;
- $[\alpha]$ -injectivity: $[\alpha]A_0 = [\alpha]A_1 \Rightarrow \Gamma, \alpha \vdash A_0 = A_1$;
- $\mathbf{U}$ -injectivity: $\mathbf{U}_l = \mathbf{U}_m \Rightarrow l = m$.

With Lemma 1, these are all the non-syntactic inputs of what follows.

7 Theorem 4.15 (StripDeriv.agda)

Theorem 3. *If $\Gamma \vdash J$ in T_R then $\text{strip}(\Gamma) \vdash \text{strip}(J)$ in T_P .*

The proof is by structural induction. The annotation congruences strip to reflexivity. strip commutes with Γ, ψ and Γ, α (strip-addC , strip-addL).

8 Lemma 4.18 without normalisation

Inversion (`RussellInversion.agda`). In a loop-free context, each typable term former has a principal type P with $P \leq T$ for every type T of the term, where $P \leq T$ means one of:

- $\Gamma \vdash P = T$;
- $\Gamma \vdash P = \mathbb{U}_k$ and $\Gamma \vdash T = \mathbb{U}_m$ with $k \leq m$ valid.

Transitivity of $\leq$ uses $\mathbb{U}$ -injectivity. The principal types are:

- $\Gamma(i)$ for v_i ;
- $\mathbb{U}_{l+}$ for $\mathbb{U}_l$;
- $\Pi(A, B)$ for $\lambda(A, B, b)$ and $\mathbb{U}_l$ for $\Pi(A, B)$, where l is the level of the components;
- $B[a]$ for $\text{app}(A, B, c, a)$;
- $[\psi]A$ for $\langle \psi \rangle_A t$;
- $[\alpha]A$ for $\langle \alpha \rangle_A u$;
- $A[l/\alpha]$ for $t_A l$.

$\emptyset$ has no least universe: there is no level 0, and $\emptyset : \mathbb{U}_\alpha$, $\emptyset : \mathbb{U}_\beta$ for incomparable α, β . So inversion gives $\emptyset : \mathbb{U}_l$ for *some* l . The large types $[\psi]A$ and $[\alpha]A$ have no typing at all.

Coercions (`Coerce.agda`). Coercions are defined by η -expansion, with cumulativity at the leaves:

$$\begin{aligned} \text{coe}_{\Pi(A, B) \leq \Pi(A, L)}(t) &= \lambda(A, L, \text{coe}_{B \leq L}(\text{app}(A \uparrow, B \uparrow^+, t \uparrow, v_0))) \\ \text{coe}_{[\alpha]A \leq [\alpha]L}(t) &= \langle \alpha \rangle_L \text{coe}_{A \leq L}(t \uparrow_A \alpha) \\ \text{coe}_{[\psi]A \leq [\psi]L}(t) &= \langle \psi \rangle_L \text{coe}_{A \leq L}(t) \\ \text{coe}_{\mathbb{U}_k \leq \mathbb{U}_m}(t) &= t. \end{aligned}$$

Joins. $\text{Join}_\Gamma(T_0, T_1, L)$ is generated by the following cases:

- $T_i = L$;
- $T_0 = \mathbb{U}_k$, $T_1 = \mathbb{U}_m$ and $L = \mathbb{U}_p$ with $p = k \vee m$ valid;
- $T_i = \Pi(A, B_i)$, $L = \Pi(A, L')$ and $\text{Join}_{\Gamma.A}(B_0, B_1, L')$;
- $T_i = [\alpha]B_i$, $L = [\alpha]L'$ and $\text{Join}_{\Gamma, \alpha}(B_0, B_1, L')$;
- $T_i = [\psi]B_i$, $L = [\psi]L'$ and $\text{Join}_{\Gamma, \psi}(B_0, B_1, L')$.

In each case the equations $T_i = \dots$ are conversions. The universe join is the *exact* sup, so no conversion ever has to be lowered from a larger universe. Coercions are well typed and congruent. Joins are stable under term substitution, level substitution and strengthening of the constraints. Coercions compute under the eliminations:

$$\text{app}(A, L, \text{coe}_\Pi(t), a) = \text{coe}_{B[a] \leq L[a]}(\text{app}(A, B, t, a)), \quad \text{coe}_{[\alpha]}(t)_L l = \text{coe}_{A[l] \leq L[l]}(t_A l).$$

Lemma 4 (diagonal, `JoinProps.Join-diag`). *Assume $\text{Join}_\Gamma(T_0, T_1, L)$, $\Gamma \vdash T_0 = T_1$, $t_i : T_i$ and $\text{coe}_0(t_0) = \text{coe}_1(t_1) : L$. Then $\Gamma \vdash t_0 = t_1 : T_0$.*

The proof is by induction on the join, deciding loops first; in a loopy context both terms equal $\emptyset$.

- **Universe case:** $k = m$ by $\mathbb{U}$ -injectivity, so $L = \mathbb{U}_{k \vee k} = \mathbb{U}_k$.
- **Π and $[\alpha]$ cases:** injectivity, then apply both sides to v_0 (respectively to α) and use β . The induction hypothesis applies, and η concludes.
- **Guard case:** Lemma 1(1) gives $B_0 = B_1$ in Γ, ψ . In Γ, ψ , guard β removes the outer $\langle \psi \rangle$, and the induction hypothesis gives $t_0 = t_1 : B_0$. Guard η concludes: $t_0 = \langle \psi \rangle t_0 = \langle \psi \rangle t_1 = t_1$.

Removing a valid guard on top (`JoinProps.normTop`). Suppose a join of T_0, T_1 is to be inverted against a universe, a Π or a $[\alpha]$. If it is a guard join $\text{Join}([\psi]B_0, [\psi]B_1, [\psi]L)$, then ψ is valid by Lemma 1(2). The inner join is then moved from Γ, ψ to Γ , and guard β removes $\langle\psi\rangle$ from the coercion equation. This is repeated while guard joins remain on top (recursion on their number).

Lemma 5 (`main, StripUniqNF.main`). *If $\Gamma \vdash u_0 : T_0$, $\Gamma \vdash u_1 : T_1$ and $\text{strip}(u_0) = \text{strip}(u_1)$, then there is a join $\text{Join}_\Gamma(T_0, T_1, L)$ with $\Gamma \vdash \text{coe}_0(u_0) = \text{coe}_1(u_1) : L$.*

The proof is by structural induction on u_0 . At each call the context is first checked for loops; if it has one, the join is $\emptyset$ by collapse. Otherwise, inversion gives principal types, and a join of the principal types is moved to T_0, T_1 along $\leq$ (`Join-sub`). The cases for v_i , U_l , Π , λ and `app` are as in `ERT/sec43.tex`. The new ones:

- $\emptyset$: the universe join $U_{l_0 \vee l_1}$ of the two universes given by inversion.
- $\langle\psi\rangle_{A_i} t_i$: apply the induction hypothesis in Γ, ψ to get $\text{Join}_{\Gamma, \psi}(A_0, A_1, L')$, which gives the guard join. The coercion equation holds because $\langle\psi\rangle_A t = t$ in Γ, ψ .
- $\langle\alpha\rangle_{A_i} u_i$: apply the induction hypothesis in Γ, α , which gives the $[\alpha]$ -join, using level β at α .
- $t_i A_i l$: apply the induction hypothesis to t_0, t_1 , then `normTop`.
 - A conversion gives $A_0 = A_1$ by $[\alpha]$ -injectivity.
 - An $[\alpha]$ -join $[\alpha]L'$ is instantiated at l (the join is moved along l/α), with the coercions computing under level application.
 - Universe and Π joins are excluded by no-confusion.
- `app`: after `normTop`, the join of the function types is a conversion or a Π -join, handled as in the ERT notes.
- $[\psi]A$ and $[\alpha]A$ have no typing.

Types are compared by a companion function `tyMain`, by induction on the type. Π of types (possibly large), $[\psi]$ and $[\alpha]$ are handled by their components. A type that lives in a universe is compared by `main` and a universe join.

Theorem 6 (4.18, no normalisation). *If $\Gamma \vdash u_0 : A_0$, $\Gamma \vdash u_1 : A_1$, $\text{strip}(u_0) = \text{strip}(u_1)$ and $\Gamma \vdash A_0 = A_1$, then $\Gamma \vdash u_0 = u_1 : A_0$ (*term-uniq-conv-NF*). If $\Gamma \vdash A_0$ type, $\Gamma \vdash A_1$ type and $\text{strip}A_0 = \text{strip}A_1$, then $\Gamma \vdash A_0 = A_1$ (*type-uniq-NF*). Together these give Lemma 4.18 (*term-uniq-NF*).*

`StripUniqNFTest.agda` checks Sterbac’s counterexample to the paper’s proof of Lemma 4.17, with levels:

$$u_0 = \text{app}(U_{\alpha^+}, U_{\alpha^+}, \lambda(U_{\alpha^+}, U_{\alpha^+}, v_0), U_{\alpha}), \quad u_1 = \text{app}(U_{\alpha^+}, U_{\alpha^{++}}, \lambda(U_{\alpha^+}, U_{\alpha^{++}}, v_0), U_{\alpha}).$$

It checks $\vdash u_0 = u_1 : U_{\alpha^{++}}$, and also $\langle\psi\rangle u_0 = \langle\psi\rangle u_1 : [\psi]U_{\alpha^{++}}$ for an arbitrary (possibly invalid) ψ and $\langle\alpha\rangle u_0 = \langle\alpha\rangle u_1 : [\alpha]U_{\alpha^{++}}$.

9 Theorem 4.19 (`SectionR.agda`)

Theorem 7. *If $\Gamma \vdash J$ in T_P , then J lifts into every well-formed T_R context Γ' with $\text{strip}(\Gamma') = \Gamma$: there is J' in Γ' with $\text{strip}(J') = J$ (*lift-**). Every well-formed T_P context has such a Γ' (*lift-WfCtx*), and the lift is unique up to conversion (Lemma 4.18).*

The proof is by structural induction on the T_P derivation. The annotations are read off the lifted premises:

- $\langle\psi\rangle$ and $\langle\alpha\rangle$ lift into Γ', ψ and Γ', α ;
- level application uses the lift of $\Gamma, \alpha \vdash A$ type;
- the premise $A[l/\alpha] = A[l'/\alpha]$ of level-equality invariance is realigned by Lemma 4.18.

Whenever two premises share a stripped term or type, their lifts are reconciled by Lemma 4.18. Levels, validity, loops and the constraint equivalence $\psi \Leftrightarrow \psi'$ are unchanged by `strip`.

10 Files

PTyping	T_P
GrdNoConf	Lemma 1 (guard injectivity / no-confusion)
StripDeriv	Theorem 4.15
RussellInversion	principal types, $\leq$
Coerce	coercions, joins, stability
JoinProps	diagonal lemma, <code>normTop</code> , <code>Join-sub</code>
StripUniqNF	Lemma 4.18
SectionR	Theorem 4.19
StripUniqNFTest	Gap417 with levels, under $\langle\psi\rangle$ and $\langle\alpha\rangle$

Guard η is `conv-GLam-eta` in `RussellTyping`, `TarskiTyping` and `PTyping`; its paper-premise forms are `mk-conv-GLam-eta`.